

MI01 – TP n° 3: VHDL séquentiel, comptage du temps

Automne 2023

1 Introduction

Dans le précédent TP, nous avons étudié l'utilité d'une machine à état pour « contrôler » une séquence d'événements. L'objectif de ce TP est de compléter cette étude par l'application d'une machine à états au comptage du temps, avec ou sans prise en compte d'entrées extérieures (hormis l'horloge).

2 Mise en place

Créez un nouveau projet dans Vivado. Configurez ce projet pour le FPGA « xc7a35tcpg236-1 » et ajoutez le fichier de contraintes `mi01.xdc`. Vous pouvez utiliser un projet différent pour chaque exercice ou utiliser le même projet. Dans ce cas, n'oubliez pas de spécifier à chaque fois l'entité que vous voulez simuler et synthétiser en utilisant l'option « Set as Top » **à la fois dans les « Design sources » et dans les « Simulation sources »**.

Attention à respecter les noms de signaux et leur casse dans votre VHDL pour que le synthétiseur fasse correctement le lien entre les signaux du modèle VHDL les signaux physiques.

3 Travail à réaliser

Exercice 1 : Diviseur de fréquence

La carte d'expérimentation de TP est équipée d'un générateur d'horloge cadencé à 100 MHz. Le signal associé est `CLK`. Les temps considérés dans les exercices suivants étant de l'ordre de la seconde, il est utile de générer un signal d'horloge interne à une fréquence plus faible (de l'ordre du hertz). À cette fin, on peut utiliser un diviseur de fréquence dont le modèle (Listing 1) et un exemple d'utilisation (Listing 2) sont donnés ci-après. L'exemple instancie un diviseur de fréquence et le connecte aux signaux d'entrée et de sortie par une approche structurelle. Le but de cet exercice est tester le diviseur de fréquence et d'en comprendre le fonctionnement pour le réutiliser dans les exercices suivants.

- Q1. — Téléchargez depuis Moodle les deux fichiers source VHDL `clock_divider.vhd` et `test_divider.vhd`.
 - Ajoutez ces deux fichiers à votre projet.
 - Désignez le module `test_divider` comme module principal s'il ne l'est pas déjà.
- Q2. Expliquez succinctement le fonctionnement du diviseur de fréquence.
- Q3. D'après la fréquence d'entrée et la valeur du diviseur dans l'exemple, quelle est la fréquence du signal de sortie ?
- Q4. Programmez le FPGA et vérifiez le fonctionnement du diviseur.

```

1  -- Diviseur de fréquence à reset asynchrone.
2  -- Divise la fréquence du signal clock_in par divisor
3  --  $f(\text{clock\_out}) = f(\text{clock\_in}) / (\text{divisor})$ 
4  entity clock_divider is
5      -- La clause generic permet de définir des paramètres pour l'entité qui
6      -- pourront être modifiés lors de l'instanciation du composant.
7      generic(divisor : integer := 100000000);
8      port(clock_in, reset : in bit; clock_out : out bit);
9  end clock_divider;
10
11 architecture Behavioral of clock_divider is
12 begin
13     process(clock_in, reset)
14         variable c : integer range 0 to divisor - 1 := 0;
15         variable clock_sync : bit;
16     begin
17         if reset = '1' then
18             c := 0;
19             clock_sync := '0';
20         elsif clock_in'event and clock_in = '1' then
21             -- Gestion de l'état du compteur de temps
22             if (c < divisor - 1) then
23                 c := c + 1;
24             else
25                 c := 0;
26             end if;
27
28             -- Calcul de l'état du signal divisé
29             if c < (divisor / 2) then
30                 clock_sync := '1';
31             else
32                 clock_sync := '0';
33             end if;
34         end if;
35         -- Affectation de la sortie
36         clock_out <= clock_sync;
37     end process;
38 end Behavioral;

```

Listing 1 – Diviseur de fréquence

```

1  -- Exemple d'instanciation du diviseur de fréquence.
2  entity test_divider is
3      port (CLK, BTN_C : in bit; LED_0 : out bit);
4  end test_divider;
5
6  architecture Behavioral of test_divider is
7      component clock_divider
8          generic(divisor : integer);
9          port(clock_in, reset : in bit; clock_out : out bit);
10     end component clock_divider;
11 begin
12     CD : clock_divider
13         -- generic map permet l'affectation d'une valeur
14         -- aux paramètres du composant. Ici divisor prend la
15         -- valeur 20000000 dans l'instance CD.
16         generic map(divisor => 20000000)
17         port map(clock_in => CLK, reset => BTN_C, clock_out => LED_0);
18 end Behavioral;

```

Listing 2 – Instanciation du diviseur de fréquence

Exercice 2 : Feux de circulation

Dans cet exercice, vous allez réaliser un contrôleur synchrone pour les feux d'un carrefour à deux axes principaux (Figure 1). Les deux axes sont symétriques, la durée de la phase rouge est de 10 secondes, la durée de la phase orange est de 2 secondes.

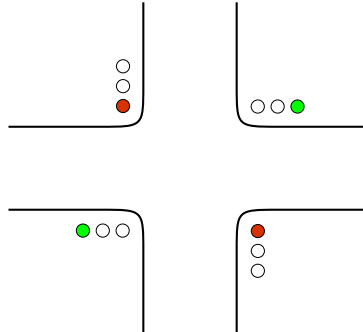


Figure 1 – Carrefour à deux axes

Vous utiliserez les signaux suivants dans votre VHDL :

CLK	Entrée	Horloge 100 MHz
BTN_C	Entrée	Signal de <i>reset</i> (le cas échéant)
LED_3_0	Sortie	Feux de l'axe 1 (Ordre : Rouge Orange Vert)
LED_7_4	Sortie	Feux de l'axe 2 (Ordre : Rouge Orange Vert)

- Q1. Définissez la durée de toutes les phases et le déroulement de la séquence.
- Q2. En fonction des durées des phases, quelle devrait être la fréquence minimale de l'horloge ?
- Q3. Modélisez le contrôleur en VHDL. Pour générer le signal d'horloge approprié, vousinstancierez un diviseur de fréquence comme dans l'exemple de l'exercice 1.
- Q4. Testez votre modèle en simulation. Vous pouvez directement forcer le signal de sortie du diviseur pour générer l'horloge d'entrée de votre système, inutile de forcer le signal CLK avec une horloge à 100 MHz.
- Q5. Programmez le FPGA et vérifiez le fonctionnement.
- Q6. Comment peut-on intégrer une remise à zéro fiable du système ?
- Q7. Codez, simulez, programmez le FPGA et testez votre solution.

Exercice 3 : Prise en compte d'un capteur de voiture

Dans une deuxième étape, on veut prendre en considération deux capteurs de véhicules installés sur les deux axes au droit des feux (boucles d'induction par exemple). Le principe de fonctionnement du carrefour devient le suivant :

- En cas d'absence de véhicule sur l'axe 2, le feu reste vert sur l'axe 1.
- Si un véhicule se présente au feu de l'axe 2, l'axe 1 passe à l'orange puis au rouge **une fois la durée minimale du vert respectée**. L'axe 2 passe au vert et le reste jusqu'à ce qu'un véhicule se présente au feu de l'axe 1.
- L'axe 2 passe à l'orange puis au rouge **une fois la durée minimale du vert respectée**. L'axe 1 passe au vert et le reste jusqu'à ce qu'un véhicule se présente au feu de l'axe 2.
- Et ainsi de suite.

On ajoute les signaux suivants au système :

BTN_G	Entrée	Bouton poussoir de gauche	Capteur de l'axe 1
BTN_H	Entrée	Bouton poussoir du haut	Capteur de l'axe 2

- Q1. Définissez le déroulement de la nouvelle séquence.
- Q2. Modifiez votre VHDL en conséquence, sans prise en compte du *reset*.
- Q3. Testez votre modèle en simulation.
- Q4. Programmez le FPGA et vérifiez le fonctionnement.
- Q5. Quel type de *reset* (synchrone ou asynchrone) vaut-il mieux mettre en œuvre et à quel moment doit-il être pris en compte ?
- Q6. Implémentez le *reset*, programmez le FPGA et testez le fonctionnement.